



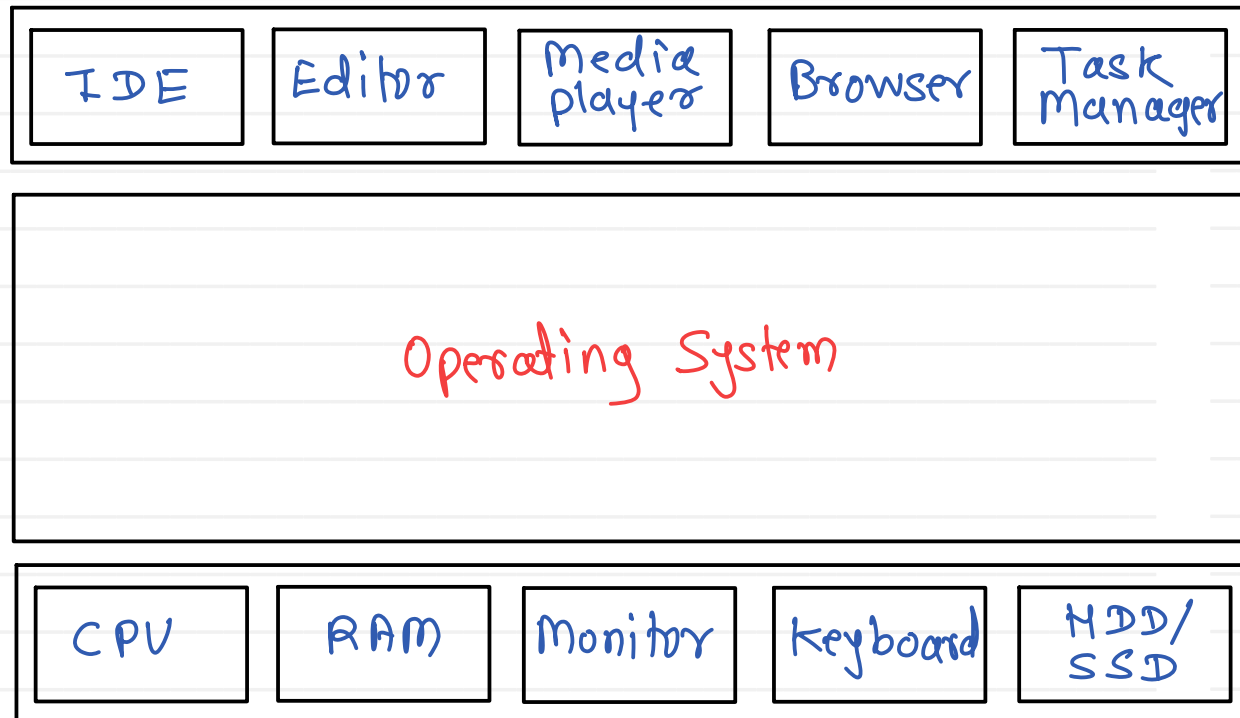
Sunbeam Institute of Information Technology
Pune and Karad

Module - Operating System Concepts and Linux Programming

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

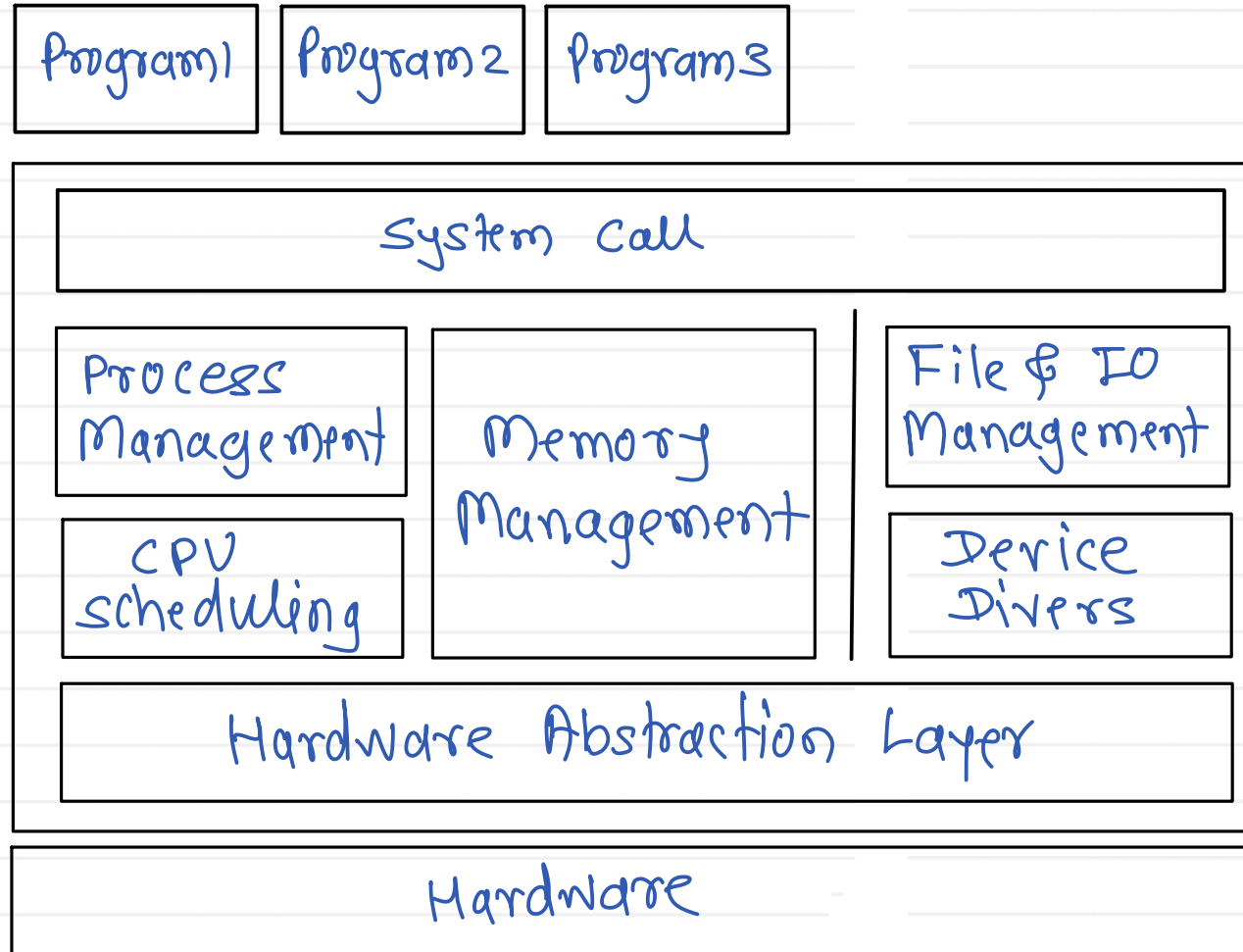
End
user



- interface betⁿ hardware & end user.
- interface betⁿ hardware & application s/w
- control program which controls execution of all the programs running system
- resource allocator/manager which manages limited h/w resources.

CD/DVD/ISO - Core OS + Applⁿ S/W + System Utilities
(kernel)

Kernel functions



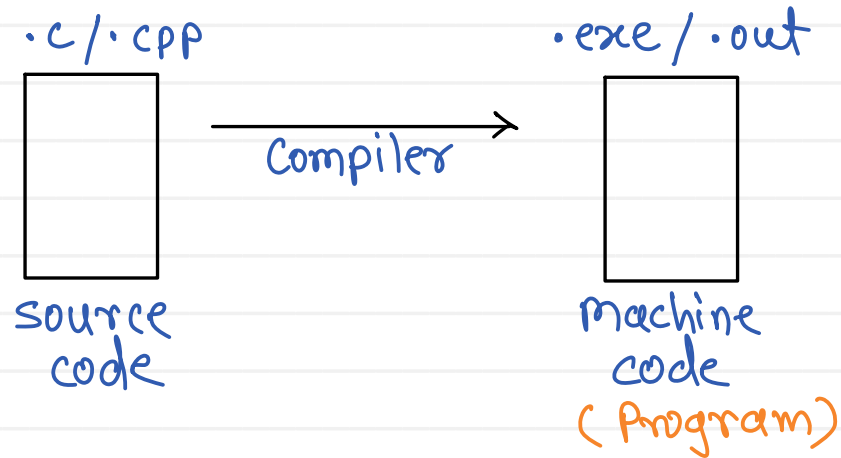
- Networking
- User Interfacing
- Security & Protection

Program compilation steps

GCC - GNU C Compiler
GNU - GNU NOT UNIX

Process : Program in execution

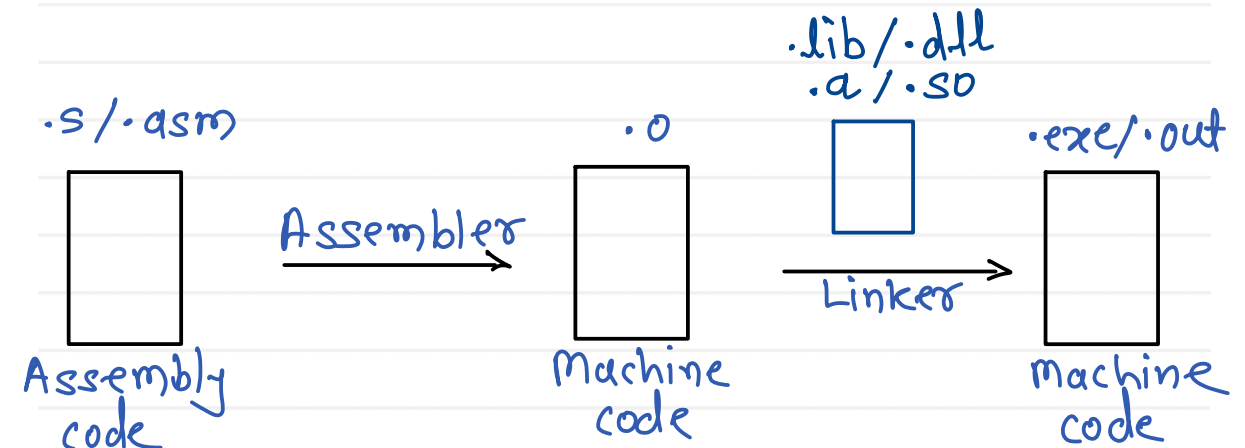
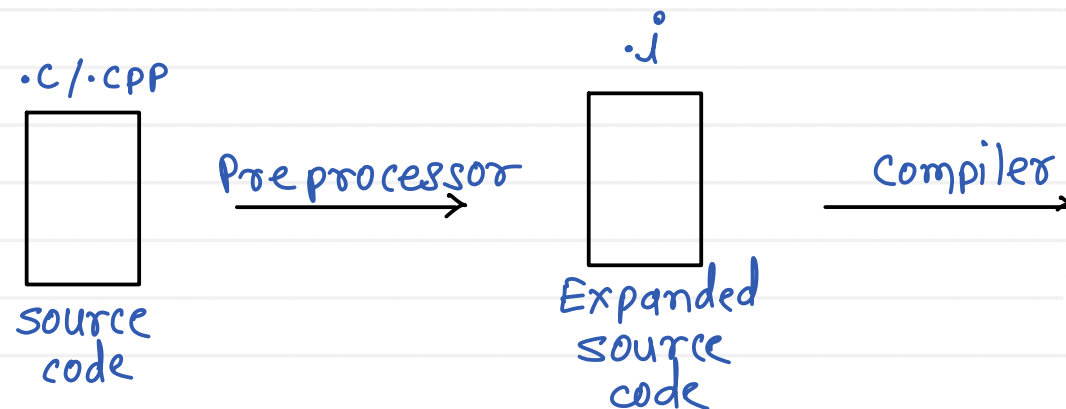
Program : set of instructions to machine (CPU)



Toolchain : (gcc)

- set of tools which work on source code one by one to convert it into machine code.

1. Preprocessor (cpp)
2. Compiler (cc)
3. Assembler (as)
4. Linker (ld)
5. Debugger (gdb)
6. Utilities (make, objdump)



•class → 0xCAFE BAFE

•exe/•out

- information required to execute a program

1. Magic number: (2 or 4 bytes)
(identity to file format)

Windows: •exe → Portable Executable → MZ

Linux: •out → Executable Linking Format
↳ ? ELF

2. info of remaining sections
(start, end, size)

3. type of a program
(CLI / GUI / Library)

4. address of entry point function (Sectioned binary) symbols



→ instructions of a program in machine code format

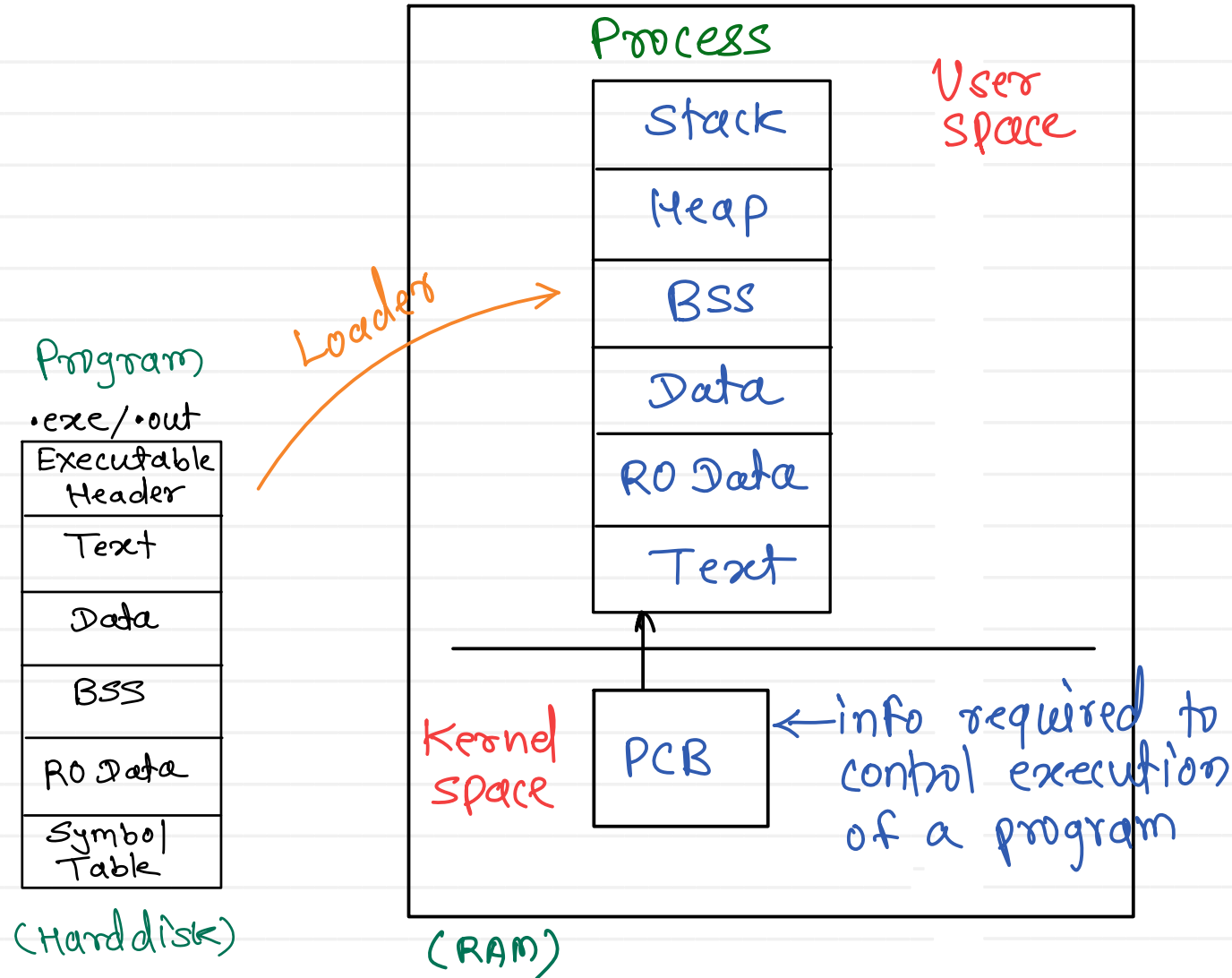
→ static & global variables (initialized)

→ static & global variables (uninitialized)

→ read only data (string constant)

→ information about symbols

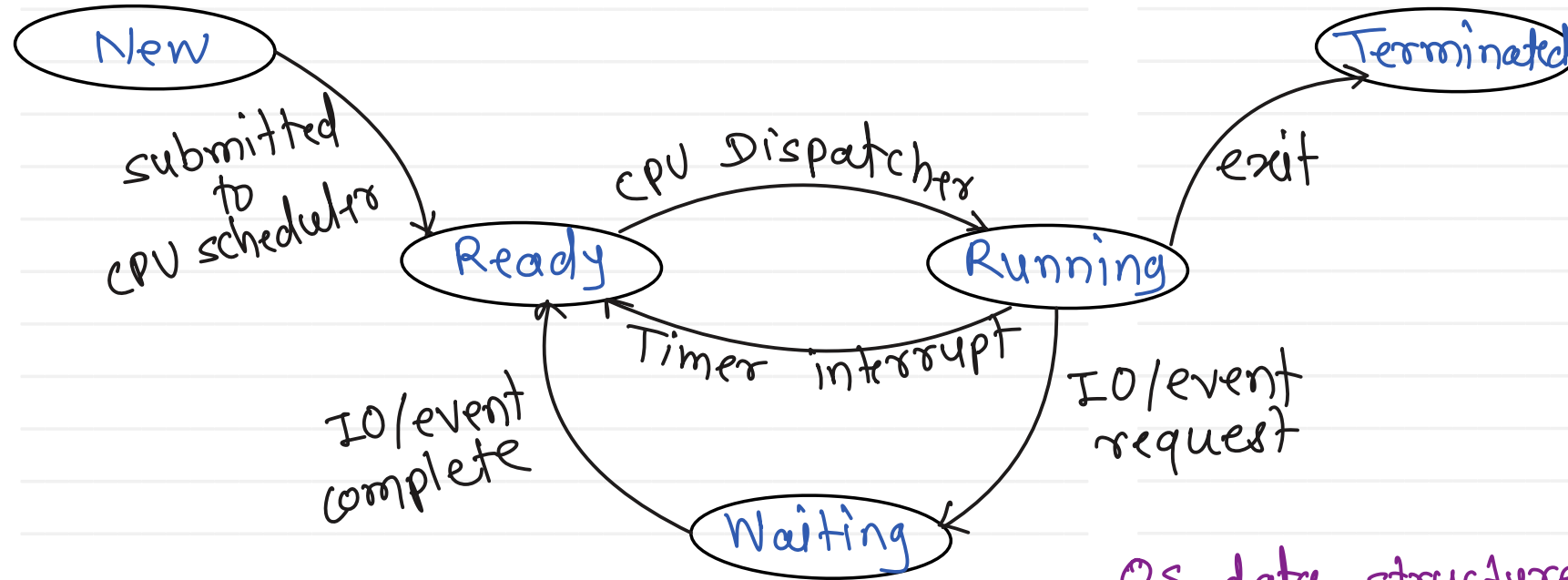
variables
functions



stack - FARs are stored
 ↳ return addr, local variables
 Heap - dynamically allocated space

PCB - process descriptor
 ↓
 struct task_struct
 (sched.h)

- pid, ppid
- exit status
- Mem info
- Sched info
- IPC info
- Open file info
- execution context
- Kernel stack



OS data structures:

1. Job queue / Process list :
 - all processes of system
2. Ready queue :
 - processes which are ready to execute on CPU
3. Waiting queues :
 - processes which are waiting for IO/event

Scheduler will be called in below cases:

1. Running $\rightarrow$ Terminated } voluntarily
2. Running $\rightarrow$ Waiting
3. Running $\rightarrow$ Ready } forcefully
4. Waiting $\rightarrow$ Ready

CPU scheduling algorithm:

1. FCFS
2. SJF (SRTF)
3. Priority
4. RR
5. CFS

Types of scheduling:

1. Pre emptive -

CPU access is given to next process forcefully.

2. Non pre emptive -

CPU access is always given to next process voluntarily.

CPU scheduling criteria's

1. CPU Utilization : (Max)

- how much CPU is busy / idle e.g. CPU Utilization = 70%

Busy = 70% & Idle = 30%

Server OS - 90% CPU Utilization

Desktop OS - 70% CPU Utilization

2. Through put : (Max)

- amount of work done in unit time
- number of processes completed in unit time

$$TAT = \text{CPU waiting time} + \text{CPU burst} + \text{IO waiting time} + \text{IO burst}$$

3. Response time : (min)

- time from arrival of process into ready queue upto first time scheduled on CPU

$$RT = ST - AT$$

4. Waiting time : (min)

- total time spent by process in ready queue

$$WT = TAT - \text{CPU burst}$$

5. Turn Around Time (TAT) : (min)

- total time spent by process inside memory (RAM)
- time from arrival of process inside memory upto departure of process from memory.

$$TAT = DT - AT$$

Execution context:

values of CPU registers

↑
small memory element which are present inside CPU

1. General purpose registers - A, B, C, D..
2. Flag register
3. Segment registers - SS, DS, CS, ES----
4. Offset registers - SP, BP, SI, DI
5. Program Counter - PC/IR

- CPU dispatcher loads execution context of process from its PCB to CPU.

Context switch:

- changing process of CPU.
- execution context of running process is saved first & then execution context of selected process is restored on CPU

